

DEEPSKILLING – JAVA FSE REACT

PL/SQL Exercises

Name: DARSHAN DONTIBOINA

SUPERSET ID: 8001653

```
CREATE TABLE Customers (  
  CustomerID INT PRIMARY KEY,  
  Name VARCHAR(100),  
  DOB DATE,  
  Balance INT,  
  LastModified DATE  
);
```

```
CREATE TABLE Accounts (  
  AccountID INT PRIMARY KEY,  
  CustomerID INT,  
  AccountType VARCHAR(20),  
  Balance INT,  
  LastModified DATE,  
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Transactions (  
  TransactionID INT PRIMARY KEY,  
  AccountID INT,  
  TransactionDate DATE,  
  Amount INT,  
  TransactionType VARCHAR(10),  
  FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);
```

```
CREATE TABLE Loans (  
  LoanID INT PRIMARY KEY,  
  CustomerID INT,  
  LoanAmount INT,  
  InterestRate INT,  
  StartDate DATE,  
  EndDate DATE,  
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Employees (  
  EmployeeID INT PRIMARY KEY,  
  Name VARCHAR(100),  
  Position VARCHAR(50),  
  Salary INT,  
  Department VARCHAR(50),  
  HireDate DATE  
);
```

```
ALTER TABLE Customers ADD IsVIP VARCHAR(5);
```

```
-- Customers
```

```
INSERT INTO Customers VALUES (1,'Amit Sharma',TO_DATE('1985-02-10','YYYY-MM-DD'),1200,SYSDATE,NULL);
```

```
INSERT INTO Customers VALUES (2,'Bhavana Reddy',TO_DATE('1990-07-20','YYYY-MM-DD'),1500,SYSDATE,NULL);
```

SUPERSET ID: 8001653

```

INSERT INTO Customers VALUES (3,'Chandan Gupta',TO_DATE('1982-11-02','YYYY-MM-DD'),2000,SYSDATE,NULL);
INSERT INTO Customers VALUES (4,'Deepa Nair',TO_DATE('1978-01-25','YYYY-MM-DD'),2500,SYSDATE,NULL);
INSERT INTO Customers VALUES (5,'Eshan Verma',TO_DATE('1995-09-10','YYYY-MM-DD'),1800,SYSDATE,NULL);

```

-- Accounts

```

INSERT INTO Accounts VALUES (1,1,'Savings',1200,SYSDATE);
INSERT INTO Accounts VALUES (2,2,'Checking',1500,SYSDATE);
INSERT INTO Accounts VALUES (3,3,'Savings',2000,SYSDATE);
INSERT INTO Accounts VALUES (4,4,'Checking',2500,SYSDATE);
INSERT INTO Accounts VALUES (5,5,'Savings',1800,SYSDATE);

```

-- Transactions

```

INSERT INTO Transactions VALUES (1,1,SYSDATE,500,'Deposit');
INSERT INTO Transactions VALUES (2,2,SYSDATE,300,'Withdrawal');
INSERT INTO Transactions VALUES (3,3,SYSDATE,700,'Deposit');
INSERT INTO Transactions VALUES (4,4,SYSDATE,400,'Withdrawal');
INSERT INTO Transactions VALUES (5,5,SYSDATE,600,'Deposit');

```

-- Loans

```

INSERT INTO Loans VALUES (1,1,5000,5,SYSDATE,ADD_MONTHS(SYSDATE,60));
INSERT INTO Loans VALUES (2,2,8000,6,SYSDATE,ADD_MONTHS(SYSDATE,48));
INSERT INTO Loans VALUES (3,3,7500,4,SYSDATE,ADD_MONTHS(SYSDATE,36));
INSERT INTO Loans VALUES (4,4,12000,7,SYSDATE,ADD_MONTHS(SYSDATE,72));
INSERT INTO Loans VALUES (5,5,9000,5,SYSDATE,ADD_MONTHS(SYSDATE,60));

```

-- Employees

```

INSERT INTO Employees VALUES (1,'Anita Iyer','Manager',70000,'HR',TO_DATE('2015-06-15','YYYY-MM-DD'));
INSERT INTO Employees VALUES (2,'Bharat Kumar','Developer',60000,'IT',TO_DATE('2017-03-20','YYYY-MM-DD'));
INSERT INTO Employees VALUES (3,'Chitra Menon','Analyst',55000,'Finance',TO_DATE('2018-09-10','YYYY-MM-DD'));
INSERT INTO Employees VALUES (4,'Devendra Singh','Tester',50000,'IT',TO_DATE('2019-12-05','YYYY-MM-DD'));
INSERT INTO Employees VALUES (5,'Esha Patil','HR Specialist',48000,'HR',TO_DATE('2020-07-22','YYYY-MM-DD'));

```

COMMIT;

-- QUESTION 1

DECLARE

```

    CURSOR cust_cur IS
        SELECT CustomerID, DOB
        FROM Customers;

```

```

    v_age NUMBER;

```

BEGIN

```

    FOR cust_rec IN cust_cur LOOP
        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, cust_rec.DOB) / 12);
        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = cust_rec.CustomerID;
        END IF;
    END LOOP;

```

```
END LOOP;  
COMMIT;  
END;  
/
```

```
select * from customers;
```

```
-- QUESTION 2
```

```
BEGIN  
  FOR cust_rec IN (  
    SELECT CustomerID, Balance  
    FROM Customers  
  ) LOOP  
  
    IF cust_rec.Balance > 10000 THEN  
      UPDATE Customers  
      SET IsVIP = 'TRUE'  
      WHERE CustomerID = cust_rec.CustomerID;  
    ELSE  
      UPDATE Customers  
      SET IsVIP = 'FALSE'  
      WHERE CustomerID = cust_rec.CustomerID;  
    END IF;  
  
  END LOOP;  
  
  COMMIT;  
END;  
/
```

```
select * from customers;
```

```
-- QUESTION 3
```

```
BEGIN  
  FOR loan_rec IN (  
    SELECT c.Name,  
           l.LoanID,  
           l.EndDate  
    FROM Customers c  
    JOIN Loans l  
    ON c.CustomerID = l.CustomerID  
    WHERE l.EndDate BETWEEN SYSDATE  
           AND SYSDATE + 30  
  ) LOOP  
  
    DBMS_OUTPUT.PUT_LINE(  
      'Reminder: Dear ' || loan_rec.Name ||  
      ', Loan ID ' || loan_rec.LoanID ||  
      ' is due on ' || loan_rec.EndDate  
    );  
  
  END LOOP;  
END;  
/
```

```
-- QUESTION 4
```

```

CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_account NUMBER,
    p_to_account NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Insufficient Funds');
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(
            'Transfer Failed: ' || SQLERRM
        );
END;
/

```

-- QUESTION 5

```

CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_employee_id NUMBER,
    p_percentage NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_percentage / 100)
    WHERE EmployeeID = p_employee_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN

```

```

        DBMS_OUTPUT.PUT_LINE(
            'Employee ID not found.'
        );

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/

-- QUESTION 6
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_customerid NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
)
IS
BEGIN
    INSERT INTO Customers
    VALUES (
        p_customerid,
        p_name,
        p_dob,
        p_balance,
        SYSDATE
    );

    COMMIT;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE(
            'Customer already exists.'
        );

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/

-- QUESTION 7
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
END;
/

```

```
select * FROM accounts;
```

```
-- QUESTION 8
```

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    p_department VARCHAR2,  
    p_bonus_percent NUMBER  
)  
IS  
BEGIN  
    UPDATE Employees  
    SET Salary = Salary +  
        (Salary * p_bonus_percent / 100)  
    WHERE Department = p_department;  
  
    COMMIT;  
END;  
/
```

```
select * from employees;
```

```
-- QUESTION 9
```

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
    p_from_account NUMBER,  
    p_to_account NUMBER,  
    p_amount NUMBER  
)  
IS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_balance  
    FROM Accounts  
    WHERE AccountID = p_from_account;  
  
    IF v_balance >= p_amount THEN  
  
        UPDATE Accounts  
        SET Balance = Balance - p_amount  
        WHERE AccountID = p_from_account;  
  
        UPDATE Accounts  
        SET Balance = Balance + p_amount  
        WHERE AccountID = p_to_account;  
  
        COMMIT;  
  
    ELSE  
        DBMS_OUTPUT.PUT_LINE(  
            'Insufficient balance.'  
        );  
    END IF;  
END;  
/
```

```
-- QUESTION 10
```

```
CREATE OR REPLACE FUNCTION CalculateAge(  

```

```

    p_dob DATE
)
RETURN NUMBER
IS
BEGIN
    RETURN FLOOR(
        MONTHS_BETWEEN(SYSDATE, p_dob) / 12
    );
END;
/

```

-- QUESTION 11

```

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
    p_loan_amount NUMBER,
    p_interest_rate NUMBER,
    p_years NUMBER
)
RETURN NUMBER
IS
    v_monthly_rate NUMBER;
    v_months NUMBER;
    v_emi NUMBER;
BEGIN
    v_monthly_rate := p_interest_rate / 12 / 100;
    v_months := p_years * 12;

    v_emi :=
        (p_loan_amount * v_monthly_rate *
         POWER(1 + v_monthly_rate, v_months))
        /
        (POWER(1 + v_monthly_rate, v_months) - 1);

    RETURN ROUND(v_emi, 2);
END;
/

```

-- QUESTION 12

```

CREATE OR REPLACE FUNCTION HasSufficientBalance(
    p_account_id NUMBER,
    p_amount NUMBER
)
RETURN BOOLEAN
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_account_id;

    RETURN v_balance >= p_amount;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;

```

/

-- QUESTION 13

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
```

-- QUESTION 14

```
CREATE TABLE AuditLog (
    AuditID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    TransactionID NUMBER,
    ActionDate DATE,
    ActionType VARCHAR2(50)
);
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog(
        TransactionID,
        ActionDate,
        ActionType
    )
    VALUES(
        :NEW.TransactionID,
        SYSDATE,
        'INSERT'
    );
END;
/
```

-- QUESTION 15

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = :NEW.AccountID;

    IF :NEW.TransactionType = 'Withdrawal' THEN

        IF :NEW.Amount > v_balance THEN
            RAISE_APPLICATION_ERROR(
                -20002,
                'Withdrawal exceeds balance'
            );
        END IF;
    END IF;
```



```

ELSIF :NEW.TransactionType = 'Deposit' THEN

    IF :NEW.Amount <= 0 THEN
        RAISE_APPLICATION_ERROR(
            -20003,
            'Deposit must be positive'
        );
    END IF;

END IF;

END;
/

-- QUESTION 16
DECLARE
    CURSOR GenerateMonthlyStatements IS
        SELECT AccountID,
            TransactionDate,
            Amount,
            TransactionType
        FROM Transactions
        WHERE TRUNC(TransactionDate,'MM') =
            TRUNC(SYSDATE,'MM');

    v_rec GenerateMonthlyStatements%ROWTYPE;
BEGIN
    OPEN GenerateMonthlyStatements;

    LOOP
        FETCH GenerateMonthlyStatements INTO v_rec;
        EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Account: ' || v_rec.AccountID ||
            ', Date: ' || v_rec.TransactionDate ||
            ', Amount: ' || v_rec.Amount ||
            ', Type: ' || v_rec.TransactionType
        );

    END LOOP;

    CLOSE GenerateMonthlyStatements;
END;
/

-- QUESTION 17
DECLARE
    CURSOR ApplyAnnualFee IS
        SELECT AccountID
        FROM Accounts;

    v_fee NUMBER := 100;
BEGIN
    FOR acc_rec IN ApplyAnnualFee LOOP

        UPDATE Accounts

```

```

        SET Balance = Balance - v_fee
        WHERE AccountID = acc_rec.AccountID;

    END LOOP;

    COMMIT;
END;
/

-- QUESTION 18
DECLARE
    CURSOR UpdateLoanInterestRates IS
        SELECT LoanID,
               InterestRate
        FROM Loans;

BEGIN
    FOR loan_rec IN UpdateLoanInterestRates LOOP

        UPDATE Loans
        SET InterestRate = loan_rec.InterestRate + 0.5
        WHERE LoanID = loan_rec.LoanID;

    END LOOP;

    COMMIT;
END;
/

-- QUESTION 19
CREATE OR REPLACE PACKAGE CustomerManagement AS

    PROCEDURE AddCustomer(
        p_customerid NUMBER,
        p_name VARCHAR2,
        p_dob DATE,
        p_balance NUMBER
    );

    PROCEDURE UpdateCustomer(
        p_customerid NUMBER,
        p_name VARCHAR2
    );

    FUNCTION GetCustomerBalance(
        p_customerid NUMBER
    ) RETURN NUMBER;

END CustomerManagement;
/

-- QUESTION 20
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(
        p_customerid NUMBER,

```

```

        p_name VARCHAR2,
        p_dob DATE,
        p_balance NUMBER
    ) IS
BEGIN
    INSERT INTO Customers
    VALUES(
        p_customerid,
        p_name,
        p_dob,
        p_balance,
        SYSDATE
    );
END;

PROCEDURE UpdateCustomer(
    p_customerid NUMBER,
    p_name VARCHAR2
) IS
BEGIN
    UPDATE Customers
    SET Name = p_name
    WHERE CustomerID = p_customerid;
END;

FUNCTION GetCustomerBalance(
    p_customerid NUMBER
) RETURN NUMBER
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Customers
    WHERE CustomerID = p_customerid;

    RETURN v_balance;
END;

END CustomerManagement;
/

-- QUESTION 21
CREATE OR REPLACE PACKAGE EmployeeManagement AS

    PROCEDURE HireEmployee(
        p_id NUMBER,
        p_name VARCHAR2,
        p_position VARCHAR2,
        p_salary NUMBER,
        p_department VARCHAR2
    );

    PROCEDURE UpdateEmployee(
        p_id NUMBER,
        p_salary NUMBER

```

```

);

FUNCTION AnnualSalary(
    p_id NUMBER
) RETURN NUMBER;

END EmployeeManagement;
/

-- QUESTION 22
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee(
        p_id NUMBER,
        p_name VARCHAR2,
        p_position VARCHAR2,
        p_salary NUMBER,
        p_department VARCHAR2
    ) IS
    BEGIN
        INSERT INTO Employees
        VALUES(
            p_id,
            p_name,
            p_position,
            p_salary,
            p_department,
            SYSDATE
        );
    END;

    PROCEDURE UpdateEmployee(
        p_id NUMBER,
        p_salary NUMBER
    ) IS
    BEGIN
        UPDATE Employees
        SET Salary = p_salary
        WHERE EmployeeID = p_id;
    END;

    FUNCTION AnnualSalary(
        p_id NUMBER
    ) RETURN NUMBER
    IS
        v_salary NUMBER;
    BEGIN
        SELECT Salary
        INTO v_salary
        FROM Employees
        WHERE EmployeeID = p_id;

        RETURN v_salary * 12;
    END;

END EmployeeManagement;

```

/

-- QUESTION 23

CREATE OR REPLACE PACKAGE EmployeeManagement AS

```
PROCEDURE HireEmployee(  
    p_id NUMBER,  
    p_name VARCHAR2,  
    p_position VARCHAR2,  
    p_salary NUMBER,  
    p_department VARCHAR2  
);
```

```
PROCEDURE UpdateEmployee(  
    p_id NUMBER,  
    p_salary NUMBER  
);
```

```
FUNCTION AnnualSalary(  
    p_id NUMBER  
) RETURN NUMBER;
```

END EmployeeManagement;

/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

```
PROCEDURE HireEmployee(  
    p_id NUMBER,  
    p_name VARCHAR2,  
    p_position VARCHAR2,  
    p_salary NUMBER,  
    p_department VARCHAR2  
) IS  
BEGIN  
    INSERT INTO Employees  
    VALUES(  
        p_id,  
        p_name,  
        p_position,  
        p_salary,  
        p_department,  
        SYSDATE  
    );  
END;
```

```
PROCEDURE UpdateEmployee(  
    p_id NUMBER,  
    p_salary NUMBER  
) IS  
BEGIN  
    UPDATE Employees  
    SET Salary = p_salary  
    WHERE EmployeeID = p_id;  
END;
```

```
FUNCTION AnnualSalary(  

```

```

        p_id NUMBER
    ) RETURN NUMBER
IS
    v_salary NUMBER;
BEGIN
    SELECT Salary
    INTO v_salary
    FROM Employees
    WHERE EmployeeID = p_id;

    RETURN v_salary * 12;
END;

END EmployeeManagement;
/

-- QUESTION 24
CREATE OR REPLACE PACKAGE AccountOperations AS

    PROCEDURE OpenAccount(
        p_accountid NUMBER,
        p_customerid NUMBER,
        p_type VARCHAR2,
        p_balance NUMBER
    );

    PROCEDURE CloseAccount(
        p_accountid NUMBER
    );

    FUNCTION GetTotalBalance(
        p_customerid NUMBER
    ) RETURN NUMBER;

END AccountOperations;
/
CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount(
        p_accountid NUMBER,
        p_customerid NUMBER,
        p_type VARCHAR2,
        p_balance NUMBER
    ) IS
    BEGIN
        INSERT INTO Accounts
        VALUES(
            p_accountid,
            p_customerid,
            p_type,
            p_balance,
            SYSDATE
        );
    END;

    PROCEDURE CloseAccount(

```

```

        p_accountid NUMBER
    ) IS
BEGIN
    DELETE FROM Accounts
    WHERE AccountID = p_accountid;
END;

FUNCTION GetTotalBalance(
    p_customerid NUMBER
) RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT SUM(Balance)
    INTO v_total
    FROM Accounts
    WHERE CustomerID = p_customerid;

    RETURN NVL(v_total,0);
END;

END AccountOperations;
/

```

OUTPUTS:

Status : Successfully executed

Time:
0 secs

Memory:
0 Mb

Your Output

Table created.

Table created.

Table created.

Status : Successfully executed

Time:

0 secs

Memory:

0 Mb

Your Output

Table altered.

1 row created.

1 row created.

Status : Successfully executed

Time:

0 secs

Memory:

0 Mb

Your Output

1 row created.

Commit complete.

PL/SQL procedure successfully completed.

OTHER OUTPUTS:

Table created.

Table created.

Table created.

Table created.

Table created.

SUPERSET ID: 8001653

Table altered.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

Commit complete.

PL/SQL procedure successfully completed.

CUSTOMERID

NAME

DOB BALANCE LASTMODIF ISVIP

1

Amit Sharma

10-FEB-85 1200 19-JUN-26

2

Bhavana Reddy

20-JUL-90 1500 19-JUN-26

CUSTOMERID

NAME

DOB BALANCE LASTMODIF ISVIP

3

Chandan Gupta

02-NOV-82 2000 19-JUN-26

4

Deepa Nair

CUSTOMERID

NAME

DOB BALANCE LASTMODIF ISVIP

25-JAN-78 2500 19-JUN-26

5
Eshan Verma
10-SEP-95 1800 19-JUN-26

PL/SQL procedure successfully completed.

CUSTOMERID

NAME

DOB BALANCE LASTMODIF ISVIP

1
Amit Sharma
10-FEB-85 1200 19-JUN-26 FALSE

2
Bhavana Reddy
20-JUL-90 1500 19-JUN-26 FALSE

CUSTOMERID

NAME

DOB BALANCE LASTMODIF ISVIP

3
Chandan Gupta
02-NOV-82 2000 19-JUN-26 FALSE

4
Deepa Nair

CUSTOMERID

NAME

DOB BALANCE LASTMODIF ISVIP

25-JAN-78 2500 19-JUN-26 FALSE

5
Eshan Verma
10-SEP-95 1800 19-JUN-26 FALSE

SUPERSET ID: 8001653

PL/SQL procedure successfully completed.

Procedure created.

Procedure created.

Warning: Procedure created with compilation errors.

Procedure created.

ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIF
1	1	Savings	1200	19-JUN-26
2	2	Checking	1500	19-JUN-26
3	3	Savings	2000	19-JUN-26
4	4	Checking	2500	19-JUN-26
5	5	Savings	1800	19-JUN-26

Procedure created.

EMPLOYEEID

NAME

POSITION

SALARY

DEPARTMENT

HIREDATE

1			
Anita Iyer			
Manager		70000	
HR		15-JUN-15	

EM...